

JORVERA

Agile Execution Plan & Weekly Sprint Backlog

A corrected, task-specific operating plan for a six-person team

Framework	Scrum with a Kanban-style work board
Sprint cadence	One working week per Sprint
Team	Six members; named primary owner, active co-owner and reviewer for each task
Delivery sequence	Walking skeleton → feature complete → frozen baseline → measured optimization → optional LoRA/QLoRA → final evaluation
Core control principle	AI proposes and explains; approved evidence, deterministic policy and human decisions control readiness and export

How to use this plan

- Before each Sprint: confirm the goal, available capacity, dependencies, acceptance criteria and demo evidence, select only work the team can realistically finish.
- During the Sprint: keep Jira/board status current, coordinate daily, integrate continuously and surface blockers immediately. Do not silently add scope that puts the Sprint Goal at risk.
- At the end of the Sprint: demonstrate a usable increment, inspect evidence against acceptance criteria, review the next priorities, and record one or two process improvements.
- For Sprint 13: the LoRA/QLoRA tasks are conditional. If Sprint 12 results in No-Go, use that week for hardening, evaluation, failure analysis and defense preparation instead of training.
- For research: preserve raw outputs and version dataset splits, prompts, model settings, policies, schemas, evaluation code and run metadata. Pilot results guide improvement; held-out results are reserved for final evaluation.

Plan at a glance

Phase	Sprints	Outcome	Exit signal
Foundation	S0–S1	A clean runnable environment and one end-to-end change path.	Walking skeleton works from baseline to initial JSON package.
Core evidence & reasoning	S2–S6	Immutable evidence, retrieval, state/transition validity, gaps and requirements.	Outputs are evidence-linked, schema-valid and reviewable.
Impact & readiness	S7–S8	Authority-aware impact graph, deterministic gate and security boundaries.	Proposed edges cannot silently propagate; missing approvals block readiness.
UI, package & hardening	S9–S10	Human review views, engineering handoff artifacts, reliable/offline behavior.	Feature-complete MVP passes end-to-end and security regression tests.
Evaluation & optimization	S11–S14	Frozen reference, controlled optimization, optional tuning and final research.	Independent rerun reproduces the release and the reported results.

1. Leadership and responsibilities

Member	Technical leadership role	Permanent responsibility
Youssef	Product and Architecture Lead	Architecture, evidence/provenance, integration contracts, package, release coordination
Mina Maged	AI and Retrieval Technical Lead	Retrieval, reranking, shared model interface, context management, AI optimization, optional LoRA/QLoRA
Neama	State and Requirements Intelligence Lead	AS-IS/TO-BE, claim verification, Transition-Level Validity, requirements assurance
Merna	Gap, Clarification and Review-Experience Lead	Coverage schema, gaps, risks, clarification, annotations, domain review flows
Mina Gerges	Control, Graph and Security Lead	Baseline versioning, impact graph, authority, STALE, gate, RBAC, isolation
Mazen	Platform, Evaluation and UI Lead	Evaluation, statistics, CI/CD, Docker, observability, shared UI platform, reproducibility

2. Critical subsystem ownership

Subsystem	Primary owner	Active co-owner	Reviewer
Architecture and API contracts	Youssef	Mina Maged	Mina Gerges
Evidence/provenance	Youssef	Mazen	Neama
Baseline/versioning	Mina Gerges	Youssef	Mazen
Retrieval/reranking	Mina Maged	Merna	Youssef
Shared model/context layer	Mina Maged	Neama	Youssef
AS-IS/TO-BE and claim verification	Neama	Mina Maged	Youssef
Transition-Level Validity	Neama	Mina Gerges	Mina Maged
Gap coverage and risks	Merna	Neama	Mina Maged
Clarification/re-analysis	Merna	Neama	Mazen
Requirements assurance	Neama	Merna	Youssef
Impact graph/edge authority	Mina Gerges	Mina Maged	Youssef
STALE/readiness gate	Mina Gerges	Mazen	Youssef
Human Review Interface	Mazen	Relevant domain owner	Youssef
Change Package/export	Youssef	Mazen	Neama
Security/isolation	Mina Gerges	Youssef	Mina Maged
Evaluation/statistics	Mazen	Mina Maged	Neama
Dataset/annotation	Merna	Neama	Mazen
LoRA/QLoRA experiment	Mina Maged	Neama	Mazen

3. Scrum and board operating model

Scrum accountabilities

- Product Owner accountability: Youssef is the proposed product-priority lead, with priority and acceptance validation from the project supervisor/stakeholder. The Product Owner orders the Product Backlog by value and maintains the Product Goal.
- Scrum Master accountability: name one team member in Jira at the start of S0. This person facilitates the Scrum events, helps remove impediments and protects effective Scrum practice. Keep the accountability explicit; do not assume it is automatically included in a technical lead title.
- Developers: all six members are responsible collectively for a usable increment. Technical subsystem leads coordinate implementation, while the team owns the Sprint Goal and Definition of Done.

Board workflow

Product Backlog → Ready → In Progress → Code Review → Test/Domain Review → Done. Use a Blocked flag with a named blocker and next action; do not create an additional workflow column for every exception. Keep work items small, link dependent items and attach the acceptance evidence before moving an item to Done.

Suggested working agreement: limit each person to at most two active implementation items at once unless the team explicitly agrees otherwise; pair on risky shared contracts; integrate small pull requests frequently; do not merge code without passing applicable tests and review.

One-week Sprint cadence (five working days)

When	Event / focus	Required output
Day 1	Sprint Planning	Sprint Goal, selected backlog items, realistic capacity, acceptance criteria, dependencies and demo evidence.
Days 1–5	Daily Scrum (15 minutes)	Inspect progress toward the Sprint Goal; update the plan and surface blockers. This is a coordination event, not a manager status report.
Day 2–3	Build and integrate	Small working paths, early tests, reviewed API/schema contracts and updated board status.
Day 3	Midweek integration checkpoint	Resolve contract mismatches, unblock dependencies and reduce scope if the goal is at risk.
Day 4	Verification and demo rehearsal	Tests, domain review, security checks relevant to the change, documentation and stable demo data.
Day 5	Sprint Review + Retrospective	Demonstrated increment, acceptance decision, stakeholder feedback, updated backlog and one or two agreed process actions.

Definition of Ready (team working agreement)

- The outcome and user/research value are understood.
- Acceptance criteria and evidence to demonstrate completion are written.
- Dependencies and affected contracts are identified.
- A Primary Owner, Active Co-owner and Reviewer are named.
- The item is small enough for the weekly Sprint, or has been split.

Definition of Done

- Acceptance criteria pass and evidence is linked to the task in Jira.
- Primary and Active Co-owner both contributed substantively and can explain/demonstrate the subsystem.
- Applicable unit, integration, security and evaluation tests exist and pass; CI passes.
- Qualified code/design review is complete; schemas, API contracts, migrations and error handling are updated.
- Upstream/downstream integration and role authorization are tested where applicable.
- Logging, runbook, technical documentation and board status are current.
- No open critical defect remains; unresolved non-critical defects are recorded and accepted explicitly.
- The increment can be demonstrated during Sprint Review.
- Research items record dataset/split version, model ID, prompt/configuration, raw outputs, metrics, cost and runtime.

4. Sprint-by-sprint task schedule

The order below preserves the source plan's workstreams and task IDs. Each sprint is one working week. The sprint content is a forecast: Sprint Planning may re-order or split work, but must protect a clear Sprint Goal and preserve the mandatory critical path. Every detailed row now maps directly to its task ID; there are no generic copy-forward execution recipes.

Sprint 0 — Professional Foundation

Sprint Goal: Create a safe delivery environment that every member can run and contribute to.

Sprint Review evidence: *A clean clone starts the API, database, mock model adapter and tests through Docker.*

Task / owner	Ownership	Implementation steps	Acceptance evidence
S0-01 **Define architecture v1 and API conventions**	Primary: Youssef Co-owner: Mina Maged Reviewer: Mina Gerges	Map service boundaries, data ownership, request flow, failure responses and API versioning. Record material architecture choices as short ADRs and publish request/response examples.	Approved architecture/API notes exist; service owners confirm boundaries and a sample request/response matches the contract.
S0-02 **Create repository, Jira project, PR/issue templates**	Primary: Mazen Co-owner: Youssef Reviewer: Mina Maged	Create the repository and Jira project; establish branch/PR conventions, task labels, issue templates, review rules and a small example issue that travels through the board.	A sample issue and PR can be created, reviewed, tested and closed using the published templates and board workflow.
S0-03 **Create FastAPI skeleton and health endpoint**	Primary: Youssef Co-owner: Mina Gerges Reviewer: Mazen	Create the FastAPI app, configuration handling, liveness/readiness endpoints and basic error model. Package the service in Docker and add startup and endpoint tests.	A fresh Docker start succeeds; liveness/readiness and expected failure tests pass.
S0-04 **Configure PostgreSQL, migrations and pgvector**	Primary: Mina Gerges Co-owner: Mina Maged Reviewer: Youssef	Configure PostgreSQL, migrations and pgvector. Add a clean database initialization path and verify extension availability, migration upgrade and rollback.	A clean database setup and migration test confirms PostgreSQL and pgvector are available.
S0-05 **Create shared model-adapter skeleton**	Primary: Mina Maged Co-owner: Neama Reviewer: Youssef	Define a provider-neutral model request/response contract. Implement a deterministic mock adapter first, schema validation, timeout handling and a switchable configured provider.	The mock adapter returns schema-valid output; invalid output, timeout and provider switching are tested.
S0-06 **Draft StateClaim and TransitionRecord schemas**	Primary: Neama Co-owner: Mina Maged Reviewer: Youssef	Define versioned StateClaim and TransitionRecord fields, enums, IDs, evidence/version references and validation rules; add valid, boundary and invalid fixtures.	Schemas validate fixtures. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S0-07 **Draft coverage schema, scenario template, annotation guide and six-pilot plan**	Primary: Merna Co-owner: Neama Reviewer: Mazen	Specify the coverage dimensions, scenario format, annotation instructions, adjudication process and six pilot scenarios. Make label definitions usable by a second annotator.	Coverage dimensions and labels are unambiguous enough for two annotators to apply to the same pilot case.
S0-08 **Configure Docker Compose, CI, logging and test framework**	Primary: Mazen Co-owner: Mina Gerges Reviewer: Youssef	Provide one Docker Compose command for app and dependencies; run formatting/linting and tests in CI; add structured logs and document common setup failures.	A fresh clone can be launched from documented steps and CI passes without machine-specific manual fixes.
S0-09 **Create scenario, gold-label and experiment-result schemas**	Primary: Mazen Co-owner: Merna Reviewer: Neama	Define versioned scenario, gold-label and experiment-result schemas, including scenario IDs, split, condition, model/configuration, predictions, metrics, cost and runtime.	Schemas validate pilot fixtures. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S0-10 **Define model candidates, token logger, API cost ceiling and offline fallback**	Primary: Mina Maged Co-owner: Mazen Reviewer: Youssef	Compare candidate model options against requirements and budget; define token/cost/latency logging, maximum spend, deterministic mock behavior and an offline-demo fallback.	The approved plan contains exact model/provider candidates, per-call logging, a numeric cost ceiling and a tested offline fallback.

Exit gate: no feature work should depend on local, undocumented setup. Shared schemas/contracts are versioned before Sprint 1 integration.

Sprint 1 — End-to-End Walking Skeleton

Sprint Goal: Pass a simple change through the entire pipeline and export an initial JSON package.

Sprint Review evidence: *A refund-change scenario runs from approved baseline through review/gating to a JSON package without manual database edits.*

Task / owner	Ownership	Implementation steps	Acceptance evidence
S1-01 **Store simple evidence and approve baseline**	Primary: Youssef Co-owner: Mina Gerges Reviewer: Mazen	Create a candidate baseline from source evidence, review and approve it, then store an immutable snapshot with manifest/hash and explicit version state.	An approved baseline snapshot is immutable; creating a later version does not modify the prior manifest/hash.
S1-02 **Submit and persist change request**	Primary: Merna Co-owner: Neama Reviewer: Youssef	Define the change-request input contract and fixture; implement validation, persistence, project association and a retrievable status/ID.	Change Request API and fixture. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S1-03 **Retrieve basic baseline context**	Primary: Mina Maged Co-owner: Merna Reviewer: Youssef	Retrieve baseline evidence for a change request and return ranked evidence IDs with source locators, scores and version/project filters.	Ranked context list. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S1-04 **Generate basic AS-IS/TO-BE and transition record**	Primary: Neama Co-owner: Mina Maged Reviewer: Mina Gerges	Generate atomic current-state and target-state claims plus a Transition Record. Link every claim to evidence and a baseline version; do not silently invent unsupported claims.	Each generated claim links to evidence/version and unsupported claims are marked insufficient or held for review.
S1-05 **Generate one gap, clarification and requirement**	Primary: Merna Co-owner: Neama Reviewer: Mazen	Produce at least one traceable gap, a clarification tied to that gap, and a candidate requirement. Preserve links among the request, evidence, question and requirement.	Linked gap/question/requirement. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S1-06 **Create one impact candidate and simple gate**	Primary: Mina Gerges Co-owner: Mazen Reviewer: Youssef	Create a minimal impact candidate and evaluate the agreed simple gate deterministically. Return BLOCK or ALLOW with explicit reason codes and the input versions used.	Gate result is deterministic for the same fixture and returns explicit reason codes for both BLOCK and ALLOW.
S1-07 **Build initial JSON Change Package**	Primary: Youssef Co-owner: Mazen Reviewer: Neama	Define a versioned JSON Change Package schema and serialize the walking-skeleton output, including baseline/change IDs, requirements, evidence links, gate status and decisions.	Versioned JSON export. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S1-08 **Create minimal UI and end-to-end integration test**	Primary: Mazen Co-owner: All domain owners Reviewer: Youssef	Build the smallest UI flow to submit a fixture, view processing status and export JSON. Add an end-to-end test that exercises the API and persistence path.	A single automated end-to-end test submits a scenario through the UI/API and downloads a valid JSON package.
S1-09 **Implement Condition A and Condition B-lite**	Primary: Mina Maged Co-owner: Neama Reviewer: Mazen	Implement Conditions A and B-lite exactly as defined in the approved research protocol. Freeze the same scenario inputs, output schema and run configuration for both.	Both conditions run against versioned, identical fixture inputs according to the approved protocol; outputs/configuration are saved.
S1-10 **Build minimal evaluator and run six pilot scenarios**	Primary: Mazen Co-owner: All members Reviewer: Youssef	Build the smallest evaluator that executes the six pilot scenarios, stores raw outputs and computes the pilot metrics, runtime, token use/cost and failure summary.	All six pilots produce a run record and a reproducible summary of metrics, cost/runtime and failures.
S1-11 **Author and independently annotate six pilot scenarios**	Primary: Merna Co-owner: All members Reviewer: Mazen	Author six bounded pilot scenarios; have two annotators label them independently, adjudicate disagreements, and version both original labels and adjudicated gold labels.	Every pilot has two independent labels, an adjudicated outcome and versioned evidence of disagreements.

Keep the first path intentionally thin. The purpose is end-to-end proof and pilot instrumentation, not production completeness.

Sprint 2 — Evidence, Baseline and Version Safety

Sprint Goal: Make evidence immutable and baseline changes explicit and versioned.

Sprint Review evidence: *The team demonstrates immutable evidence, an approved frozen v1 baseline and a separate v2 change.*

Task / owner	Ownership	Implementation steps	Acceptance evidence
S2-01 **Implement DOCX, PDF, Markdown and text parsers**	Primary: Youssef Co-owner: Mazen Reviewer: Neama	Parse DOCX, PDF, Markdown and plain text. Preserve source metadata and page/heading/paragraph locators; test empty, malformed, duplicate and bilingual files.	Parser fixtures cover every supported format, source locators are retained, and malformed files fail safely.
S2-02 **Implement segmentation and chunking baseline**	Primary: Youssef Co-owner: Mina Maged Reviewer: Merna	Create deterministic segmentation/chunking that preserves headings, table/rule boundaries and source positions. Give segments stable IDs and test repeatability.	Deterministic evidence segments. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S2-03 **Generate evidence IDs, locators, hashes and provenance**	Primary: Youssef Co-owner: Mazen Reviewer: Mina Gerges	Assign project-scoped evidence IDs; store source/segment hashes, locators, provenance and baseline/version links. Reject incomplete references and verify stored records are immutable.	Hashes and project scoping are validated; the audit trail can reconstruct each evidence record's source.
S2-04 **Reject unknown and cross-project evidence references**	Primary: Mina Gerges Co-owner: Youssef Reviewer: Neama	Validate every evidence reference at API and persistence boundaries. Reject unknown IDs, cross-project IDs and references inconsistent with the selected baseline.	Unknown and cross-project evidence references are rejected in automated negative tests.
S2-05 **Implement candidate baseline approval and frozen snapshots**	Primary: Mina Gerges Co-owner: Youssef Reviewer: Mazen	Implement candidate, review and approval states for baselines. Freeze approved content into a snapshot with a manifest and hashes.	Approved snapshot contains a manifest/hash and cannot be edited in place.
S2-06 **Implement new baseline versions and no-silent-mutation rule**	Primary: Mina Gerges Co-owner: Youssef Reviewer: Mazen	Create later baseline versions by explicit predecessor links; prove v2 can be added without changing the v1 content, hashes or approval record.	A v2 baseline is created with a predecessor link while all v1 content and hashes remain unchanged.
S2-07 **Review bilingual source segmentation quality**	Primary: Merna Co-owner: Neama Reviewer: Mina Maged	Review source segmentation on representative Arabic-English/code-switched cases; annotate splits, lost context, ordering and locator errors, then fix high-impact issues.	Representative Arabic-English fixtures have documented segmentation review outcomes and fixed high-impact defects.
S2-08 **Add audit and migration tests**	Primary: Mina Gerges Co-owner: Mazen Reviewer: Youssef	Test audit event creation and database migration paths, including clean install, upgrade, rollback, baseline history and preservation of evidence/version relationships.	Audit and migration suite passes. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.

The immutability rule applies to approved evidence and baselines. Corrections create new versions; they do not silently rewrite history.

Sprint 3 — Retrieval and Shared AI Infrastructure

Sprint Goal: Provide reproducible ranked evidence through one controlled model layer.

Sprint Review evidence: *A fixed query set produces ranked evidence, metrics, saved model inputs and latency/cost logs.*

Task / owner	Ownership	Implementation steps	Acceptance evidence
S3-01 **Implement lexical and pgvector retrieval**	Primary: Mina Maged Co-owner: Merna Reviewer: Youssef	Build lexical search and pgvector semantic retrieval behind one interface; return stable evidence IDs, locators and scores for a fixed set of queries.	A fixed query set returns both lexical and semantic ranked evidence lists with stable IDs/locators.

Task / owner	Ownership	Implementation steps	Acceptance evidence
S3-02 **Implement hybrid fusion and change-aware reranking**	Primary: Mina Maged Co-owner: Merna Reviewer: Mazen	Fuse lexical and semantic candidate lists, then rerank for the change context. Keep the fusion/reranking configuration versioned for repeatable comparison.	Hybrid ranking is reproducible from the same configuration and is evaluated on frozen relevance labels.
S3-03 **Add entity, alias, module and artifact filters**	Primary: Mina Maged Co-owner: Merna Reviewer: Neama	Add filters for entities, aliases, modules and artifact types. Test positive matches, aliases, irrelevant artifacts and combinations of filters.	Filtered relevant results. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S3-04 **Add dependency and version relevance signals**	Primary: Mina Maged Co-owner: Mina Gerges Reviewer: Youssef	Incorporate dependency links and baseline/version compatibility into ranking or filtering. Test cases where a newer or unrelated version must not dominate.	Version-aware ranking. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S3-05 **Implement shared model interface and context budget**	Primary: Mina Maged Co-owner: Neama Reviewer: Youssef	Implement one provider-neutral model invocation layer with schema contracts, token/context budget, timeout/retry policy and captured model/config metadata.	All model calls pass through one interface and log model/config, tokens, latency, cost and failure state.
S3-06 **Define state-oriented context requirements**	Primary: Neama Co-owner: Mina Maged Reviewer: Merna	Specify which state claims require which evidence types, ordering and minimum context. Define behavior for missing, conflicting and over-budget evidence.	Context contract documented. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S3-07 **Create relevance annotations and bilingual queries**	Primary: Merna Co-owner: Neama Reviewer: Mazen	Create bilingual retrieval queries and relevance labels, including difficult terminology and hard negatives; version the query set and label guide.	The gold set has versioned bilingual queries, relevance labels and adjudication for disagreements.
S3-08 **Implement Recall@K, MRR, nDCG and latency tracking**	Primary: Mazen Co-owner: Mina Maged Reviewer: Merna	Implement reproducible Recall@K, MRR, nDCG and latency calculations over a frozen query set; save inputs, outputs, metric code version and results.	Metric results can be regenerated from saved predictions and a versioned evaluation script.

Use a fixed query/relevance set for comparisons. Keep retrieval relevance separate from downstream generation quality.

Sprint 4 — State Alignment and Transition-Level Validity

Sprint Goal: Produce evidence-backed states and a controlled transition status with reason codes.

Sprint Review evidence: *Supported claims can still yield an invalid or unresolved transition when version, rule, exception or authority conditions fail.*

Task / owner	Ownership	Implementation steps	Acceptance evidence
S4-01 **Finalize StateClaim and TransitionRecord schemas**	Primary: Neama Co-owner: Mina Maged Reviewer: Youssef	Finalize the schemas based on walking-skeleton integration. Version required fields, enums, status definitions, evidence links and migration examples.	Versioned schemas. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S4-02 **Generate AS-IS/TO-BE with evidence references**	Primary: Neama Co-owner: Mina Maged Reviewer: Youssef	Generate evidence-linked AS-IS/TO-BE claims at atomic granularity, with source spans and baseline versions; preserve missing/contradictory evidence rather than smoothing it over.	Evidence-linked claims. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S4-03 **Implement SUPPORTED/CONTRADICTED/INSUFFICIENT and abstention**	Primary: Neama Co-owner: Youssef Reviewer: Merna	Implement SUPPORTED, CONTRADICTED and INSUFFICIENT labels plus abstention. Validate classifications against labeled fixtures and ensure insufficient evidence is not called supported.	Tests distinguish all three support labels and confirm unsupported/unclear claims are not silently marked SUPPORTED.
S4-04 **Implement model schema validation, repair and one retry**	Primary: Mina Maged Co-owner: Neama Reviewer: Mazen	Validate model output against schemas; repair a malformed response at most once, then return a typed failure. Log the original error without accepting invalid output silently.	Malformed output is repaired at most once; if still invalid, the stage returns a typed failure without accepting it.

Task / owner	Ownership	Implementation steps	Acceptance evidence
S4-05 **Implement live-rule, version, exception and authority checks**	Primary: Mina Gerges Co-owner: Neama Reviewer: Youssef	Implement deterministic checks for applicable rules, baseline version, exception scope, temporal applicability and decision authority. Keep rule results separate from LLM suggestions.	Deterministic transition checks cover rule, version, exception, temporal and authority cases.
S4-06 **Map validity statuses and reason codes**	Primary: Neama Co-owner: Mina Gerges Reviewer: Mazen	Map rule outcomes to VALID, INVALID or REQUIRES_REVIEW (or the exact agreed status enum) with stable reason codes and a documented precedence order.	VALID/INVALID/REQUIRES statuses. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S4-07 **Create conflict, ambiguity and supersession scenarios**	Primary: Merna Co-owner: Neama Reviewer: Mina Gerges	Create labeled fixtures for contradictions, ambiguity, supersession, expired rules, scoped exceptions and unsupported transitions; document expected outcomes.	Gold transition fixtures. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S4-08 **Run pilot Transition-Level Validity evaluation on the six pilot scenarios**	Primary: Mazen Co-owner: Neama Reviewer: Mina Maged	Run the six pilot scenarios through the transition evaluator; report per-category counts, errors and examples, and avoid treating pilot results as final evidence.	Pilot report includes per-category results and examples; it is labeled pilot/exploratory rather than final validation.
S4-09 **Demonstrate supported claims with an invalid transition**	Primary: Neama Co-owner: Mina Gerges Reviewer: Mazen	Build at least four distinct invalid-transition cases where individual AS-IS/TO-BE claims are supported but the overall transition must be rejected or held for review.	At least four distinct invalid transition categories are rejected or routed to review despite supported individual claims.

Transition validity is separate from whether individual claims are supported. Preserve that distinction in schemas, tests and reporting.

Sprint 5 — Gap Coverage, Risks and Clarification

Sprint Goal: Create structured gaps and questions that close specific blockers.

Sprint Review evidence: *A stakeholder answer becomes new evidence, closes a named blocker and reruns only affected analysis stages.*

Task / owner	Ownership	Implementation steps	Acceptance evidence
S5-01 **Finalize fixed coverage-vector schema**	Primary: Merna Co-owner: Neama Reviewer: Mina Maged	Finalize a versioned coverage vector with dimension definitions, evidence expectations and covered/missing/conflicting states; add examples and validation fixtures.	Versioned coverage schema. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S5-02 **Implement actor, behavior, conditions, permissions and data coverage**	Primary: Merna Co-owner: Mina Maged Reviewer: Neama	Evaluate actor, behavior, conditions, permissions and data coverage against baseline and change request; persist dimension-level evidence and status.	Coverage checks pass. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S5-03 **Implement validation, exceptions, integration, security, privacy, audit and NFR coverage**	Primary: Merna Co-owner: Mina Gerges Reviewer: Neama	Extend coverage checks to validation, exceptions, integration, security, privacy, audit and non-functional requirements; define evidence expected for each dimension.	Coverage checks pass. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S5-04 **Generate evidence-linked gaps and candidate risks**	Primary: Merna Co-owner: Mina Gerges Reviewer: Neama	Generate structured gap and candidate-risk records with source evidence, severity, rationale, affected dimension and blocker IDs; distinguish confirmed facts from suggestions.	Every proposed gap/risk has a source link, dimension, severity/rationale and blocker ID or is explicitly non-blocking.
S5-05 **Generate and rank gate-referenced questions**	Primary: Merna Co-owner: Neama Reviewer: Mazen	Generate questions only when answers can resolve or narrow a named blocker. Rank by criticality/information gain and retain the blocker IDs each question addresses.	Each generated clarification references an active blocker and has an explained priority; irrelevant questions are rejected by tests/review.
S5-06 **Record answers as evidence and trigger targeted re-analysis**	Primary: Youssef Co-owner: Neama Reviewer: Merna	Store stakeholder answers as new versioned evidence, connect them to the question and blocker, and rerun only the analysis stages affected by the answer.	An accepted answer is stored as new evidence and reruns only dependent stages, with prior versions retained.

Task / owner	Ownership	Implementation steps	Acceptance evidence
S5-07 **Expose blocker IDs and authority constraints**	Primary: Mina Gerges Co-owner: Merna Reviewer: Youssef	Expose blocker IDs, authority restrictions and the associated gate rules in the API/UI. Validate that every blocking decision links to an existing blocker and policy reason.	Every blocker shown to the user resolves to the same blocker referenced by its deterministic gate reason.
S5-08 **Implement annotation UI and gap/clarification metrics**	Primary: Merna Co-owner: Mazen Reviewer: Neama	Build annotation/review controls for gaps and clarifications; calculate gap precision/recall and question usefulness against adjudicated pilot labels.	Metrics are calculated against adjudicated labels and the review flow stores annotations and reviewer decisions.

A question is useful only if it is connected to a blocker and its answer can change a decision or close an evidence gap.

Sprint 6 — Requirements Assurance

Sprint Goal: Convert validated analysis into evidence-linked engineering requirements.

Sprint Review evidence: *A reviewed change produces traceable requirements and acceptance criteria linked to evidence, transitions and baseline versions.*

Task / owner	Ownership	Implementation steps	Acceptance evidence
S6-01 **Finalize requirements and traceability schemas**	Primary: Neama Co-owner: Merna Reviewer: Youssef	Define versioned requirement and traceability records, including type, testability, status, acceptance criteria and links to evidence, transition and baseline.	Versioned schemas. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S6-02 **Generate business and functional requirements**	Primary: Neama Co-owner: Merna Reviewer: Mina Maged	Generate atomic business and functional requirements from approved analysis; remove duplicates, preserve source traceability and route results through human review.	Reviewed requirements. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S6-03 **Generate NFRs, rules, constraints and acceptance criteria**	Primary: Neama Co-owner: Merna Reviewer: Youssef	Generate NFRs, rules, constraints and acceptance criteria. Require measurable or verifiable wording and label unsupported assumptions for review.	Reviewed requirements. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S6-04 **Classify NEW, REFINEMENT, CONTRADICTION and UNRELATED**	Primary: Neama Co-owner: Merna Reviewer: Mina Maged	Classify requirements as NEW, REFINEMENT, CONTRADICTION or UNRELATED against the approved baseline; test each class and ambiguous examples.	Fixtures exist for all four relationship types, with an explicit outcome for ambiguous cases.
S6-05 **Classify SUPERSESSION, EXCEPTION, TEMPORAL_CHANGE and UNRESOLVED**	Primary: Neama Co-owner: Mina Gerges Reviewer: Youssef	Classify SUPERSESSION, EXCEPTION, TEMPORAL_CHANGE and UNRESOLVED using version, authority and scope evidence; abstain when the deciding evidence is missing.	Authority/temporal classifications are traceable to source evidence; unresolved cases abstain instead of guessing.
S6-06 **Link material requirements to evidence and transition records**	Primary: Youssef Co-owner: Neama Reviewer: Mazen	Link each material requirement to the source change, supporting evidence, relevant TransitionRecord and baseline version; verify links resolve and survive export.	Every material requirement has resolvable evidence, transition and baseline links in the API and export.
S6-07 **Implement requirement model orchestration and optimization**	Primary: Mina Maged Co-owner: Neama Reviewer: Youssef	Orchestrate requirement generation through the shared model layer with fixed schemas, context budgets, failure handling and captured configuration.	Controlled model stage. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S6-08 **Build requirements review UI and fixtures**	Primary: Mazen Co-owner: Neama Reviewer: Merna	Implement approve/edit/reject actions for generated requirements; persist reviewer, time, rationale and revised content; test required fields and role permissions.	Approve/edit/reject actions persist actor, timestamp, rationale and final reviewed content.

Human approval is explicit. Generated requirements remain candidates until a reviewer accepts or edits them.

Sprint 7 — Impact Graph and Authority Propagation

Sprint Goal: Identify affected artifacts while preventing proposed edges from propagating STALE.

Sprint Review evidence: *Equivalent candidate edges have different propagation behavior based on authority status, with explanations and metrics.*

Task / owner	Ownership	Implementation steps	Acceptance evidence
S7-01 **Finalize edge schema and authority lifecycle**	Primary: Mina Gerges Co-owner: Mina Maged Reviewer: Youssef	Finalize impact-edge fields, allowed states, authority source, evidence, version and lifecycle transitions; enforce valid state changes in schema and tests.	All four edge states have valid schema definitions and tested transitions; invalid transitions are rejected.
S7-02 **Generate candidate impact edges**	Primary: Mina Maged Co-owner: Neama Reviewer: Mina Gerges	Generate proposed source-to-target impact edges with edge type, explanation, evidence references and AI_PROPOSED state; do not grant propagation authority by default.	AI_PROPOSED candidates. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S7-03 **Implement bounded graph traversal**	Primary: Mina Gerges Co-owner: Mina Maged Reviewer: Youssef	Implement bounded traversal with explicit depth and scope limits; return impacted artifact IDs and the path that led to each candidate.	Traversal respects the configured depth/scope limit and returns a traceable path for each affected artifact.
S7-04 **Implement CONFIRMED/RULE_DERIVED/AI_PROPOSED/REJECTED review**	Primary: Mina Gerges Co-owner: Mazen Reviewer: Neama	Implement review actions to confirm, mark rule-derived, retain as AI-proposed or reject an edge. Record reviewer/authority and prevent invalid state transitions.	Reviewer actions persist audit details and transition an edge only to an allowed authority state.
S7-05 **Propagate STALE only through authoritative edges**	Primary: Mina Gerges Co-owner: Youssef Reviewer: Mazen	Propagate STALE only over CONFIRMED or approved RULE_DERIVED edges according to the policy. Prove AI_PROPOSED and REJECTED edges cannot propagate STALE.	A test proves AI_PROPOSED and REJECTED edges never propagate STALE.
S7-06 **Prepare expected test and critical-impact annotations**	Primary: Merna Co-owner: Neama Reviewer: Mazen	Create expected impacted-artifact labels and critical-impact cases; annotate valid paths, excluded paths and cases where proposed edges must not count.	Gold impact set. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S7-07 **Create impact explanation and evidence links**	Primary: Neama Co-owner: Mina Maged Reviewer: Youssef	Attach a human-readable explanation and evidence/path links to each impact candidate; make it possible to inspect why an artifact was selected.	Explainable candidates. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S7-08 **Implement graph review UI and run pilot impact evaluation**	Primary: Mazen Co-owner: Mina Gerges Reviewer: Merna	Build graph review controls and evaluate the pilot gold impact set; report precision/recall and false-STALE cases with traceable examples.	Pilot impact metrics are reproducible from the gold set and false-STALE examples are documented.
S7-09 **Compare authority-aware propagation with authority-unaware traversal**	Primary: Mina Gerges Co-owner: Mazen Reviewer: Merna	Run paired authority-aware and authority-unaware traversals on identical fixtures; compare impacted sets and false-STALE outcomes with configuration held constant.	Paired comparison uses identical inputs and reports the difference in affected sets and false-STALE rate.

Candidate impact edges do not gain propagation authority merely because a model produced them.

Sprint 8 — Readiness Gate, Security and Decisions

Sprint Goal: Fail closed when evidence, authority, reviews, versions or decisions are incomplete.

Sprint Review evidence: *The system reports specific BLOCK reasons, records authorized resolutions and permits ALLOW only when deterministic policy passes.*

Task / owner	Ownership	Implementation steps	Acceptance evidence

Task / owner	Ownership	Implementation steps	Acceptance evidence
S8-01 **Define versioned gate policy and reason codes**	Primary: Mina Gerges Co-owner: Mazen Reviewer: Youssef	Encode versioned readiness rules and stable reason codes for missing evidence, invalid transitions, unreviewed blockers, stale versions and unauthorized decisions.	Policy schema and tests. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S8-02 **Implement BLOCK/ALLOW evaluator**	Primary: Mina Gerges Co-owner: Mazen Reviewer: Youssef	Implement a deterministic BLOCK/ALLOW evaluator over approved persisted records. Missing fields, policy errors or processing failures must not default to ALLOW.	Missing/invalid required inputs and processing failure return BLOCK or a non-allowing failure state; no implicit ALLOW path exists.
S8-03 **Implement RBAC and project isolation**	Primary: Mina Gerges Co-owner: Youssef Reviewer: Mazen	Define roles and allowed actions; enforce project scoping at API and query layers; test unauthorized writes/reads and cross-project leakage attempts.	Authorization tests cover allowed and denied operations, including cross-project access at API and query levels.
S8-04 **Implement stale-package and export-authorization checks**	Primary: Youssef Co-owner: Mina Gerges Reviewer: Mazen	Check current baseline/package versions and export permissions at export time; reject stale packages and unauthorized export requests with auditable reasons.	Stale-version and unauthorized export attempts are blocked and logged with actionable reasons.
S8-05 **Define evidence, transition, requirement and decision completeness rules**	Primary: Neama Co-owner: Mina Gerges Reviewer: Merna	Define completeness rules for evidence, transitions, requirements, blockers and human decisions; make the gate evaluate these as explicit checks.	Completeness validation. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S8-06 **Implement risk-acceptance and clarification-closure decisions**	Primary: Merna Co-owner: Mina Gerges Reviewer: Mazen	Record risk acceptance and clarification closure as attributable human decisions with rationale, scope and version. Never infer approval from an LLM answer.	Risk acceptance/clarification closure can be performed only by authorized humans and is auditable.
S8-07 **Implement model failure and graceful degradation paths**	Primary: Mina Maged Co-owner: Neama Reviewer: Mazen	Handle provider timeout, invalid output, retry exhaustion and dependency failure using typed PROCESSING_FAILED or equivalent states; prevent partial results from looking complete.	Provider/model failures cannot emit a package that looks complete or bypass the readiness gate.
S8-08 **Build gate UI, security tests and failure dashboard**	Primary: Mina Gerges Co-owner: Mazen Reviewer: Youssef	Expose gate status, blocking reason codes and remediation paths in the UI; test BLOCK/ALLOW cases and verify no model response can override the deterministic decision.	UI shows the exact gate reason codes, and changing LLM text cannot change the deterministic decision.

The readiness gate is deterministic and fail-closed. LLM output may explain a result but cannot override policy.

Sprint 9 — Human Review Interface and Change Package

Sprint Goal: Reach feature-complete status with full review and handoff outputs.

Sprint Review evidence: *Reviewers inspect the refund scenario end to end and export an approved current package in the agreed formats.*

Task / owner	Ownership	Implementation steps	Acceptance evidence
S9-01 **Complete shared UI shell, navigation and API client**	Primary: Mazen Co-owner: Merna Reviewer: Youssef	Complete shared navigation, view shell, API client, route/error handling and loading/empty states. Verify all planned views are reachable using the supported roles.	Seven-view UI platform. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S9-02 **Complete baseline, state and requirements review views**	Primary: Mazen Co-owner: Neama Reviewer: Youssef	Complete baseline, AS-IS/TO-BE and requirements review views, including evidence locators, version context and approve/edit/reject actions.	Domain actions work. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S9-03 **Complete gap, clarification, risk and QA review views**	Primary: Merna Co-owner: Mazen Reviewer: Neama	Complete gap, clarification, risk and QA review views; show blockers, questions, accepted answers, labels and links to their source evidence.	Domain actions work. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.

Task / owner	Ownership	Implementation steps	Acceptance evidence
S9-04 **Complete impact, supersession and readiness views**	Primary: Mina Gerges Co-owner: Mazen Reviewer: Youssef	Complete impact, supersession and readiness views; show edge authority, propagation paths, decision status and deterministic gate reasons.	Domain actions work. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S9-05 **Generate SRS, RTM and Decision Record**	Primary: Youssef Co-owner: Neama Reviewer: Mazen	Generate SRS, RTM and Decision Record from approved versioned data with stable IDs and cross-document traceability.	SRS, RTM and Decision Record are generated from the same approved package and share stable traceability IDs.
S9-06 **Generate CIA, TIA and Development Handoff**	Primary: Youssef Co-owner: Mina Gerges Reviewer: Merna	Generate CIA, TIA and Development Handoff from the reviewed impact set and requirements; include affected artifacts, test implications and unresolved blockers.	CIA, TIA and Development Handoff enumerate impacted artifacts, test implications and open blockers consistently.
S9-07 **Complete JSON, Markdown and CSV export plus audit trail**	Primary: Youssef Co-owner: Mazen Reviewer: Neama	Complete JSON, Markdown and CSV exports plus audit trail; validate field parity, stable identifiers, current version, encoding and reproducibility.	JSON, Markdown and CSV contain consistent approved data and correctly identify the package/baseline version.
S9-08 **Add AI explanation payloads and observability**	Primary: Mina Maged Co-owner: Mazen Reviewer: Neama	Expose model/provider/config version, evidence references, confidence/abstention status where supported, token use, latency and failure reason without claiming unsupported certainty.	Visible model provenance. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S9-09 **Run full end-to-end acceptance test**	Primary: All members Co-owner: Mazen Reviewer: Youssef	Run a complete stakeholder workflow from baseline and change request to human review, gate decision and export; record pass/fail results for each acceptance criterion.	All agreed end-to-end acceptance criteria pass; failed criteria are logged and not hidden by the demo.

Generate all handoff artifacts from the same approved version so cross-document links and statuses remain consistent.

Sprint 10 — Feature-Complete Hardening

Sprint Goal: Make the protected MVP stable, secure, observable, and usable offline.

Sprint Review evidence: *The protected MVP passes functional, integration, security, migration-recovery and offline-demo checks.*

Task / owner	Ownership	Implementation steps	Acceptance evidence
S10-01 **Run prompt-injection and malformed-output tests**	Primary: Neama Co-owner: Mina Maged Reviewer: Mazen	Create adversarial fixtures for prompt injection, malicious document instructions, malformed model responses and oversized inputs; assert safe typed outcomes.	Prompt-injection and malformed-output suite passes without granting the model authority to alter rules or approve results.
S10-02 **Run fabricated/cross-project evidence tests**	Primary: Youssef Co-owner: Mina Gerges Reviewer: Neama	Attempt to inject fabricated evidence IDs, unknown sources and cross-project references; verify each is rejected and no unsupported claim enters the approved package.	Fabricated and cross-project evidence attempts are rejected and absent from the approved package.
S10-03 **Run authorization, isolation and stale-export tests**	Primary: Mina Gerges Co-owner: Youssef Reviewer: Mazen	Test role authorization, project isolation and stale-export denial at both API and UI boundaries; verify negative cases and audit logs.	Unauthorized actions and stale exports are blocked in automated regression tests.
S10-04 **Add model timeouts, caching, batching and retrieval recovery**	Primary: Mina Maged Co-owner: Mazen Reviewer: Youssef	Add bounded timeouts, retry policy, caching and batching where safe; define retrieval-dependency failure behavior and compare latency/cost before and after.	Failure behavior is bounded; before/after logs document any quality, latency and cost trade-offs.
S10-05 **Finalize package schema and export consistency**	Primary: Youssef Co-owner: Mazen Reviewer: Neama	Freeze package field names and semantics, validate cross-document consistency and stable IDs, and test JSON/Markdown/CSV outputs from the same approved run.	Stable versioned package. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.

Task / owner	Ownership	Implementation steps	Acceptance evidence
S10-06 **Finalize bilingual usability, annotation guide and citation-verification matrix**	Primary: Merna Co-owner: Neama Reviewer: Mazen	Test bilingual reviewer flows; finalize annotation instructions and verify citations against authoritative publication/DOI records with claim-to-source mapping.	The review guide is checked against bilingual UI cases and citations have verified bibliographic details and claim mapping.
S10-07 **Add CI quality gates, migration recovery and performance logs**	Primary: Mina Gerges Co-owner: Mazen Reviewer: Youssef	Add CI quality gates, migration recovery checks and performance logs; demonstrate clean setup, migration and recovery paths on a fresh environment.	CI passes from a clean clone and documented migration/rollback/recovery tests pass.
S10-08 **Create offline demo mode and backup fixtures**	Primary: Merna Co-owner: Mazen Reviewer: Youssef	Prepare cached, versioned demo fixtures and an offline path; demonstrate the core workflow without network access to external model endpoints.	Core demo completes with external network disabled using versioned fixtures and no manual database edits.

Treat security, reliability and offline operation as release criteria, not optional polish.

Sprint 11 — Frozen Baseline and Initial Evaluation

Sprint Goal: Freeze the reference system and obtain pre-optimization results.

Sprint Review evidence: *The frozen configuration can be rerun and produces traceable baseline metrics and failure cases.*

Task / owner	Ownership	Implementation steps	Acceptance evidence
S11-01 **Freeze architecture contracts, policies and package version**	Primary: Youssef Co-owner: Mina Gerges Reviewer: Mazen	Tag a release candidate and freeze architecture/API contracts, policies and package schema. Record code revision, dependency lock and migration version.	Release candidate tag. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S11-02 **Freeze model, retrieval, context and decoding settings**	Primary: Mina Maged Co-owner: Neama Reviewer: Mazen	Freeze provider/model ID, retrieval configuration, context assembly, decoding and retry settings. Record why the chosen settings were selected.	Frozen AI configuration. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S11-03 **Freeze state, transition and requirements prompts/schemas**	Primary: Neama Co-owner: Mina Maged Reviewer: Youssef	Freeze state/transition/requirements prompts and schemas; version each artifact and record changes from the development configuration.	Frozen mechanism config. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S11-04 **Freeze coverage, clarification and annotation configuration**	Primary: Merna Co-owner: Neama Reviewer: Mazen	Freeze coverage rules, clarification behavior and annotation guide; ensure the guide and label schema match the evaluator.	Frozen mechanism config. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S11-05 **Freeze graph authority and gate policy**	Primary: Mina Gerges Co-owner: Youssef Reviewer: Mazen	Freeze edge authority lifecycle, STALE propagation rules and readiness policy; capture policy version/hash and associated test results.	Frozen control config. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S11-06 **Freeze dataset splits and A-D protocol**	Primary: Mazen Co-owner: All members Reviewer: Youssef	Create immutable pilot/development/held-out manifests; check duplicates and leakage, lock evaluation protocol and protect held-out labels from tuning.	Pilot/dev/held-out manifests are immutable; duplicate/leakage checks pass and held-out labels are inaccessible to tuning workflow.
S11-07 **Run A-D baseline and three confirmatory ablations**	Primary: Mazen Co-owner: All mechanism owners Reviewer: Neama	Run A-D and the three confirmatory ablations on the frozen initial evaluation setup; save raw outputs, configuration, metrics, cost and runtime.	Initial A-D/ablation runs have saved raw outputs, run metadata and reproducible metrics.
S11-08 **Run supporting/exploratory ablations and failure analysis**	Primary: Mina Maged Co-owner: Mazen Reviewer: Merna	Run only predeclared supporting/exploratory analyses on permitted development/pilot data; document each observed failure as an optimization backlog item.	Each exploratory result is marked exploratory and yields a ranked backlog item tied to a measured failure.

Once frozen, the reference configuration is changed only through a recorded new version. Protect held-out labels.

Sprint 12 — Non-Training Optimization

Sprint Goal: Improve measured weaknesses before considering fine-tuning.

Sprint Review evidence: *Changes are compared with the Sprint 11 frozen reference using controlled regression results.*

Task / owner	Ownership	Implementation steps	Acceptance evidence
S12-01 **Improve segmentation and evidence context contracts**	Primary: Youssef Co-owner: Mina Maged Reviewer: Merna	Improve segmentation/context rules based on documented development-set failures; compare evidence preservation and retrieval suitability without changing held-out data.	Better evidence quality. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S12-02 **Tune retrieval weights and reranker**	Primary: Mina Maged Co-owner: Merna Reviewer: Mazen	Tune retrieval weights/reranker on the development set, then rerun the frozen retrieval benchmark and report gains, regressions and latency.	Reported retrieval changes are measured on the same frozen query set; no held-out labels are used for tuning.
S12-03 **Optimize context selection, caching and model choice**	Primary: Mina Maged Co-owner: Neama Reviewer: Mazen	Compare context selection, caching and model options under the same query set; report quality, latency, token/cost impact and reproducibility.	Quality/latency/cost comparison. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S12-04 **Refine transition and requirements prompts/schemas**	Primary: Neama Co-owner: Mina Maged Reviewer: Mazen	Change transition/requirements prompts or schemas only in response to measured error categories; isolate changes and rerun a fixed regression suite.	Reduced measured failures. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S12-05 **Refine gap and clarification mechanisms**	Primary: Merna Co-owner: Neama Reviewer: Mazen	Refine gap and clarification logic against labeled development cases; report false gaps, missed blockers and whether questions resolve the intended blocker.	Reduced false gaps/questions. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S12-06 **Tune graph rules and gate policy**	Primary: Mina Gerges Co-owner: Youssef Reviewer: Mazen	Adjust graph/gate rules using labeled development cases; compare false-STALE and false-block rates and re-run all critical security/authority tests.	Controlled false-STALE/false-block rates. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S12-07 **Run controlled regression evaluation**	Primary: Mazen Co-owner: All members Reviewer: Youssef	Run a controlled before/after regression with fixed scenario inputs; report changes per mechanism, quality, cost, latency and regressions.	A before/after report shows changed variables, frozen inputs, quality, latency, cost and regression cases.
S12-08 **Write evidence-based LoRA/QLoRA Go-No-Go decision**	Primary: Mina Maged Co-owner: Mazen Reviewer: All members	Make and sign the Go/No-Go decision using predeclared evidence: repeated model-specific failure, stable MVP, frozen untuned baseline, sufficient permitted data, feasible hardware/license and protected held-out split. Do not train in this task.	A signed decision selects Go or No-Go against written criteria; no training happens before this decision.

Optimize one measured variable at a time where feasible. Do not tune on the held-out set or compare against a moving baseline.

Sprint 13 — Optional LoRA/QLoRA Experiment

Sprint Goal: Fine-tune only when the Go decision is supported by evidence.

Sprint Review evidence: *If Go criteria pass, the tuned adapter is fairly compared with the untuned reference; otherwise the week is used for hardening/evaluation.*

Conditional sprint: execute S13-01–S13-09 only after the signed Sprint 12 Go decision. For No-Go, replace this sprint with extra hardening, frozen-baseline evaluation, failure analysis, reproducibility and defense preparation; do not train an adapter.

Task / owner	Ownership	Implementation steps	Acceptance evidence

Task / owner	Ownership	Implementation steps	Acceptance evidence
S13-01 **Select target subtask, base model and success metric**	Primary: Mina Maged Co-owner: Neama Reviewer: Mazen	If Go is approved, choose one bounded target subtask, base model and primary success metric; predefine comparison, compute ceiling, stopping rule and rollback.	Bounded experiment definition. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S13-02 **Review model license and data governance**	Primary: Youssef Co-owner: Mina Maged Reviewer: Mina Gerges	Verify the base-model license, permitted training/use, data consent/privacy and redistribution constraints. Record the approved use and restrictions.	Use approval cites the exact base model license and the applicable data/privacy restrictions.
S13-03 **Prepare and quality-review training data**	Primary: Merna Co-owner: Neama Reviewer: Mina Maged	Prepare a versioned training set from permitted data; quality-check labels and duplicates and exclude protected held-out examples.	Versioned training set. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S13-04 **Protect held-out data and prevent leakage**	Primary: Merna Co-owner: Mazen Reviewer: Neama	Separate data by scenario/source before training; audit overlap and near-duplicates and retain a manifest proving held-out protection.	Leakage audit demonstrates held-out set isolation at scenario/source level and records exceptions.
S13-05 **Choose LoRA or QLoRA and freeze training config**	Primary: Mina Maged Co-owner: Mazen Reviewer: Neama	Choose LoRA or QLoRA based on hardware and model fit; freeze hyperparameters, seed, base checkpoint, training budget and evaluation procedure.	Reproducible config. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S13-06 **Train, save and version adapter metadata**	Primary: Mina Maged Co-owner: Neama Reviewer: Mazen	Run training within the approved budget; store adapter weights, training configuration, data hash, environment and model metadata for reproducibility.	Adapter artifact. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S13-07 **Integrate separate experimental endpoint and rollback**	Primary: Mina Maged Co-owner: Youssef Reviewer: Mina Gerges	Expose the tuned adapter through a separate experimental endpoint/flag; preserve the untuned endpoint and demonstrate immediate rollback.	Safe experimental deployment. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S13-08 **Compare tuned and untuned quality, latency and cost**	Primary: Mazen Co-owner: Mina Maged Reviewer: Neama	Compare tuned and untuned systems on the same protected held-out set and same metrics; report quality, latency, cost and category-level regressions.	Tuned/untuned comparison uses the same held-out examples and metrics; regressions and compute costs are reported.
S13-09 **Accept or reject adapter and document failures**	Primary: All members Co-owner: Mina Maged Reviewer: Youssef	Accept the adapter only if the predefined success rule passes without unacceptable regressions; otherwise reject it and document negative results and rollback.	Adapter is accepted only against the predeclared success rule; a No-Go/rejection includes reason and rollback evidence.

Training is optional and evidence-gated. The untuned production/reference path must remain available for comparison and rollback.

Sprint 14 — Final Evaluation and Defense

Sprint Goal: Deliver a reproducible release and defend both positive and negative results.

Sprint Review evidence: *A clean environment reproduces the final demo, evaluation outputs and defense walkthrough.*

Task / owner	Ownership	Implementation steps	Acceptance evidence
S14-01 **Run final A-D system experiment**	Primary: Mazen Co-owner: All members Reviewer: Youssef	Run the final frozen A-D protocol on the protected held-out set; capture raw predictions, dataset/config hashes, metrics, cost and runtime.	Final result files, dataset/config hashes and metrics are reproducible from the frozen protocol.
S14-02 **Run three confirmatory mechanism ablations**	Primary: Mazen Co-owner: Neama/Merna/Mina Gerges Reviewer: Youssef	Run the three confirmatory mechanism ablations under the frozen protocol; report the effect of each contribution and any interactions the design supports.	Three confirmatory ablations are completed with their planned metrics and saved paired results.

Task / owner	Ownership	Implementation steps	Acceptance evidence
S14-03 **Run supporting/exploratory ablations**	Primary: Mazen Co-owner: Mina Maged/Merna Reviewer: Neama	Run supporting analyses only as exploratory; label them clearly, retain raw outputs and do not present exploratory findings as confirmatory evidence.	Secondary results. Demonstrate from a clean/repeatable run, link the output/test evidence to the task, and have the named reviewer accept it.
S14-04 **Calculate confidence intervals, effect sizes and Holm-corrected tests**	Primary: Mazen Co-owner: Mina Maged Reviewer: Youssef	Calculate paired effect sizes, scenario-level confidence intervals and the planned Holm-corrected tests; document assumptions and treat non-significance as inconclusive.	Statistical report records effect sizes, confidence intervals, multiple-comparison correction and limitations.
S14-05 **Complete failure analysis and validity threats**	Primary: Merna Co-owner: Neama Reviewer: Mazen	Analyze representative failures, annotation disagreement and threats to internal/external validity; state that findings are bounded to the evaluated refund workflow and data.	Failure analysis includes representative cases and explicit external-validity limitations.
S14-06 **Fix final critical defects and freeze release**	Primary: All members Co-owner: Workstream leads Reviewer: Youssef	Fix only release-blocking defects; freeze code/model/prompt/policy/schema/split/container versions, hash artifacts and create a stable release tag.	Release tag maps to immutable code/model/config/dataset artifacts; no unversioned change remains.
S14-07 **Finalize Docker image, runbook and reproducibility package**	Primary: Mazen Co-owner: Youssef Reviewer: Mina Gerges	Build the Docker image and runbook; have a team member other than the primary implementer reproduce the demo and evaluation from a clean environment.	A different team member reproduces demo and evaluation in a fresh environment from the runbook.
S14-08 **Finalize thesis, presentation and end-to-end defense roles**	Primary: All members Co-owner: Youssef Reviewer: Mazen	Finalize thesis, presentation and role-based defense walkthrough; ensure every reported result traces to a saved run and all limitations are explicit.	Every thesis result links to a saved experiment; slides and defense roles cover methods, results and limitations.

Report negative and inconclusive results honestly. Reproducibility and explicit limitations are release requirements.

5. Cross-cutting dataset and annotation workstream

Dataset work begins in S0 and proceeds alongside product implementation. Do not leave scenario authoring and labeling until final evaluation. Every scenario should be bounded and versioned, and include the source baseline, change request, relevant evidence, gold state claims, transition labels, gaps, expected impacted artifacts, edge authority, gate expectations and adjudicated decisions as applicable.

Member	Dataset / annotation responsibility
Youssef	Baseline evidence, provenance, versions, decisions and final package fields.
Mina Maged	Retrieval relevance labels, hard retrieval cases, bilingual terminology and model-run metadata.
Neama	AS-IS/TO-BE gold claims, support labels, transition labels and requirements labels.
Merna	Scenario coordination, gap labels, clarification answers, annotation guide and scenario coherence.
Mina Gerges	Impact edges, authority labels, expected STALE behavior, security fixtures and gate reason codes.
Mazen	Annotation/evaluation tooling, split manifests, inter-annotator agreement, leakage checks and result format.

Bias and leakage controls: do not let the primary author of a held-out scenario act as both final annotators; hide held-out labels during prompt tuning; blind annotators to experimental condition where practicable; adjudicate disagreements using the written guide; report inter-annotator agreement before adjudication. Ask the supervisor or an independent reviewer to adjudicate a held-out subset where feasible.

6. Incremental evaluation timeline

Milestone	Required evaluation
S0	Define scenario, label, metric, result and experiment-configuration schemas.
S1	Run Conditions A and B-lite plus a minimal evaluator on six pilot scenarios.
S4	Pilot Transition-Level Validity; include supported-claim cases with invalid overall transitions.
S5	Pilot baseline-relative gap coverage and clarification metrics.
S7	Pilot authority-aware impact propagation against authority-unaware traversal.
S9	Run end-to-end feature-complete regression and package-quality evaluation.
S11	Freeze model, prompts, policies, schemas, splits and A-D protocol; run the initial frozen evaluation.
S14	Run final held-out A-D evaluation, three confirmatory ablations, supporting analyses and failure reporting.

This is a bounded graduation-scale evaluation, not proof of general effectiveness across industries or all languages. Report paired effect sizes, scenario-level confidence intervals, critical-case recall and failure analysis. Treat a statistically non-significant result as inconclusive or "not demonstrated," not as equivalence.

7. Model and compute budget controls

Configured model: Record provider, exact model ID, context window, decoding settings, selection rationale and version before any held-out evaluation.

API usage: Log input/output tokens, calls, retries, latency and cost by pipeline stage and experimental condition.

Evaluation estimate: Before each frozen run, estimate scenarios × conditions × mechanism runs × repeats × average calls per pipeline; include retry and failure overhead.

Cost ceiling: Approve a numeric maximum API budget before the S1 pilot and again before final evaluation. Stop or reduce exploratory runs before exceeding it.

Embeddings: Record embedding model ID/dimensions, indexed segment count, cache policy and regeneration cost.

Availability: Maintain versioned fixtures, deterministic mocks and an offline demo mode for endpoint/network failure.

LoRA/QLoRA: Before Go, specify base model/license, available GPU/VRAM, storage, maximum training hours/runs, success criterion and rollback method.

8. Team skills and onboarding evidence

Member	Required skills	Onboarding / exit evidence
Youssef	FastAPI, PostgreSQL, hashing, provenance, API contracts, access control and package generation	Evidence-to-package vertical slice with audit and export tests
Mina Maged	IR, embeddings, pgvector, reranking, structured model invocation, context optimization and PEFT	Frozen retrieval benchmark plus shared model adapter; optional reproducible adapter training
Neama	LLM schemas, NLP evaluation, requirements analysis, conflict/ambiguity reasoning and experiment design	State/transition evaluator and Requirements Assurance mechanism pass pilot cases
Merna	Requirements analysis, coverage modeling, annotation design, clarification, HCI and test design	Annotation guide, adjudicated scenarios and gap/clarification pilot report

Member	Required skills	Onboarding / exit evidence
Mina Gerges	SQL, recursive graph queries, versioning, RBAC, policy rules, security tests and bounded traversal	Authority graph, false-STALE tests and deterministic gate pass
Mazen	Python evaluation, statistics, CI/CD, Docker, UI integration, experiment tracking and technical writing	Independent held-out run, confidence intervals, Docker build and runbook reproduce

9. Research validity and reference controls

- External-validity boundary: findings are limited to the frozen e-commerce refund baseline, represented artifact types, tested workload and included Arabic-English change-request patterns. Do not claim general effectiveness across industries, organizations, languages, repository scales, safety-critical contexts or regulatory environments.
- Transition-Level Validity tests must include cases where AS-IS, TO-BE and change claims are individually supported but the overall transition is invalid because of version mismatch, conflict, exception scope, supersession authority, temporal applicability or unresolved interpretation.
- Before proposal/thesis submission, verify each citation using a DOI or authoritative publication page; confirm authors/year/venue/pages; map the cited source to the exact claim; and state what the source does not establish.
- Record deviations from the planned protocol. If the team changes dataset composition, metrics, prompts or model configuration after a freeze, version it and explain whether the change is exploratory or confirmatory.

10. Workload planning estimate

The percentages below are retained from the source plan. The original estimate assigns 55% weight to Primary Owner work, 30% to Active Co-owner work and 15% to Reviewer work, then divides shared work among named participants. These are rough planning estimates only; actual capacity and effort should be reviewed every Sprint.

Member	Estimated weighted workload
Youssef	16.3%
Mina Maged	16.6%
Neama	18.2%
Merna	14.8%
Mina Gerges	14.8%
Mazen	19.3%

Capacity rule: do not treat these percentages as commitments or performance targets. At Sprint Planning, subtract classes/exams and other known commitments, then select work that fits the actual week. Re-estimate after observing delivered work; never use velocity to compare individuals.

11. Sprint Planning checklist

☐ Is the Sprint Goal one clear outcome rather than a list of unrelated tasks?

☐ Do all selected items have written acceptance criteria and an identifiable demo/test?

☐ Are model, data, database, UI and integration dependencies represented on the board?

☐ Does the Sprint fit this week’s real availability for all owners and reviewers?

☐ Are risky contracts paired and integrated early enough to avoid a Day 5 surprise?

☐ Are the relevant security, privacy, cost and data-version constraints known?

☐ Is the planned result demonstrable without manual database edits or unrecorded configuration?

☐ Has any optional work been explicitly dropped before the Sprint Goal is at risk?